Phase 7
Encryption Explained

Submission 2: understanding RSA, AES, and the hybrid scheme before writing a single line of crypto code

Purpose: This document explains, from scratch, what encryption and decryption are, the difference between the two families of ciphers, what RSA actually does, and what the file crypto_keys.h holds. It assumes no prior cryptography background.

Builds on: Phase 6 (plain unencrypted voice between two devices).
What's added in Phase 7: a security layer so the voice on the LTE link is unintelligible to anyone who intercepts it.
Status: design / understanding stage — no firmware written yet.

1. What Encryption and Decryption Are

Encryption is scrambling data so that anyone who intercepts it sees only meaningless garbage. **Decryption** is the reverse — turning that garbage back into the original data.

A **key** is a secret number that controls the scrambling. The same data scrambled with two different keys produces two completely different outputs. Without the correct key, the garbage cannot be turned back into anything useful.

In this project the "data" is the stream of 8-bit audio samples. Today (Phase 6) those bytes travel the LTE link in the clear — anyone who taps the relay or the cellular link could record the conversation. After Phase 7, an interceptor sees only scrambled bytes.

```
WITHOUT encryption (Phase 6):
       "Hello"  ->  [48 65 6C 6C 6F]  ----LTE---->  [48 65 6C 6C 6F]  ->  "Hello"
                                       (readable by anyone tapping the link)

   WITH encryption (Phase 7):
       "Hello"  ->  ENCRYPT(key)  ->  [9F 2A C1 ...]  ----LTE---->  DECRYPT(key)  ->  "Hello"
                                       (garbage to anyone without the key)
```

2. Two Families of Encryption

There are two fundamentally different kinds of cipher. This project uses **one of each**, because each solves a problem the other cannot.

Property	Symmetric	Asymmetric
Number of keys	ONE shared secret key	TWO keys: public + private
Same key both ways?	Yes — same key encrypts and decrypts	No — public key encrypts, private key decrypts
Speed	Very fast	Slow
Good for	Large amounts of data (the audio stream)	Small amounts safely (delivering a key)
Example used here	AES-128	RSA-512

2.1 — Symmetric (AES): fast, but has a key-delivery problem

Symmetric encryption is fast enough to scramble thousands of audio bytes per second. Its weakness: both devices must hold the **same secret key**, and that key cannot simply be sent over the LTE link in plain text — an attacker watching the link would grab it and then decrypt everything.

2.2 — Asymmetric (RSA): solves the key-delivery problem

Asymmetric encryption is built exactly to move a secret safely across an untrusted link. It is too slow to encrypt the whole audio stream, but it is perfect for sending one small secret — the AES key — securely.

3. What RSA Is

RSA gives each device **two matching keys**:

Key	Secret?	Role
Public key	NO — can be shared with anyone, even publicly	Used to ENCRYPT a message for the owner
Private key	YES — never leaves the device	The ONLY key that can DECRYPT what the public key encrypted

The defining property of RSA:

Anything encrypted with the public key can only be decrypted with the matching private key — and nothing else, not even the public key that locked it.

3.1 — The padlock analogy

Think of the public key as an **open padlock**. You make many copies of your open padlock and hand them out to anyone who wants one — you do not care who has them. Someone puts a message in a box, snaps your padlock shut, and sends the box to you. Once the padlock clicks shut, **only the single physical key you keep** can open it again. Even the person who locked the box cannot reopen it.

```
Device A wants to send a secret to Device B:

      Device B's public key  =  an open padlock B handed out
      Device B's private key =  the one key B keeps to itself

      A:  put secret in box  ->  lock with B's padlock  ->  send box over LTE
      Interceptor: sees a locked box, has no key  ->  cannot open it
      B:  receives box  ->  opens with its private key  ->  reads the secret
```

3.2 — Why it is secure (the math, briefly)

RSA's security rests on a simple fact about numbers: multiplying two large prime numbers together is easy, but taking the result and finding which two primes produced it ("factoring") is practically impossible for large enough numbers. The public key is built from the product; the private key depends on knowing the original primes. "RSA-512" means the numbers involved are 512 bits long — roughly 155 decimal digits.

> The actual operations are: encrypted = message ^ E mod N and message = encrypted ^ D mod N, where (N, E) is the public key and (N, D) is the private key. The STM32 only has to compute these two formulas — it never has to generate the keys.

RSA-512 is considered weak by modern standards and would not be used to protect real systems. It is chosen here because it is light enough to run on the STM32F401 and is fully adequate to demonstrate the concept for an academic project.

4. The Hybrid Scheme — Using Both Together

RSA is too slow to encrypt every audio packet; AES needs a shared key it cannot safely deliver on its own. The solution, used by HTTPS, WhatsApp, and essentially all secure systems, is to **combine them**: RSA delivers the AES key, then AES does the bulk work.

```
STEP 1  Caller generates a fresh random 16-byte AES key (the "session key").

   STEP 2  Caller RSA-encrypts that small key with the PEER's PUBLIC key.   <-- RSA, once
              random AES key  ->  rsa_encrypt(peer_public)  ->  64-byte blob

   STEP 3  Caller sends the 64-byte blob over LTE.
              Safe: only the peer's private key can unwrap it.

   STEP 4  Peer RSA-decrypts the blob with its OWN PRIVATE key.
              -> recovers the exact same 16-byte AES key.

   STEP 5  Both devices now hold the SAME AES key.
           Every audio packet from now on:  aes_ctr_crypt()  before send / after receive.
                                                                  <-- AES, every packet
```

RSA runs **once per call**, where its slowness does not matter. AES runs on **every audio packet**, where its speed is exactly what is needed. A new random AES key each call means that even if one call's key were exposed, earlier and later calls stay protected.

	RSA-512	AES-128
When it runs	Once, at call setup	On every audio packet
What it protects	The 16-byte AES session key	The actual voice audio
Key used	Peer public key (encrypt), own private key (decrypt)	The shared 16-byte session key
Speed needed	Not critical	Critical — must keep up with 8 kHz audio

5. What crypto_keys.h Is

crypto_keys.h is a C header file that holds the RSA keys as arrays of numbers, compiled directly into the firmware. The STM32 cannot generate RSA keys itself — prime-number searching is far too slow on the microcontroller — so the keys are generated once on a PC by keygen.py and written into this file.

5.1 — What it contains

Symbol	Meaning
RSA_A_N	Device A's modulus N (64 bytes) — part of both its keys
RSA_A_D	Device A's private exponent D (64 bytes) — secret
RSA_B_N	Device B's modulus N (64 bytes)
RSA_B_D	Device B's private exponent D (64 bytes) — secret
RSA_E	The public exponent E, shared by both (typically 65537)
IS_DEVICE_A	A flag: 1 on device ...001, 0 on device ...002

A **public key** is the pair (N, E). A **private key** is the pair (N, D). The IS_DEVICE_A flag tells each device which set of numbers is "mine" and which belongs to "the peer" — the same idea as swapping PHONE_NUMBER per board.

5.2 — Both devices share one file

keygen.py is run **once**. The resulting crypto_keys.h is copied to BOTH devices unchanged — only the IS_DEVICE_A flag differs. If the file were regenerated separately for each device, the two would hold different keys and could never decrypt each other.

5.3 — How the device picks its keys

```
#if IS_DEVICE_A
  #define MY_N    RSA_A_N   /* my modulus  */
  #define MY_D    RSA_A_D   /* my private exponent (secret) */
  #define PEER_N  RSA_B_N   /* peer's modulus */
#else
  #define MY_N    RSA_B_N
  #define MY_D    RSA_B_D
  #define PEER_N  RSA_A_N
#endif

/* Encrypt for the peer:  uses (PEER_N, RSA_E)  -> peer public key
   Decrypt what I receive: uses (MY_N,   MY_D)  -> my private key   */
```

6. How It Fits the Voice Project

The encryption is added as a self-contained module — two new files, crypto.c and crypto.h — so the rest of the firmware barely changes. main.c simply calls into it.

6.1 — The crypto module functions

Function	Job
crypto_init()	One-time setup at startup
rsa_encrypt() / rsa_decrypt()	Wrap / unwrap the AES session key (once per call)
aes_ctr_set_key()	Load the shared 16-byte AES key
aes_ctr_crypt()	Encrypt or decrypt one audio packet (every packet)

6.2 — Where it plugs into the call

```
Caller presses CALL
        generate random 16-byte AES key
        rsa_encrypt(key with peer public key)  ->  64-byte blob
        send blob as one control message
   Peer answers
        rsa_decrypt(blob with own private key) ->  same 16-byte AES key
   Both sides now share the AES key
        outgoing audio: aes_ctr_crypt() just before tcp_send_audio()
        incoming audio: aes_ctr_crypt() just before writing play_buf
```

6.3 — Encryption is symmetric in code

AES-CTR mode has a convenient property: the **same function** both encrypts and decrypts. The sender calls aes_ctr_crypt() on plain audio to scramble it; the receiver calls the identical function on the scrambled bytes to recover the audio. No separate decrypt routine is needed for the audio stream.

7. Summary

Term	One-line meaning
Encryption	Scrambling data so interceptors see only garbage
Key	The secret number that controls the scrambling
Symmetric (AES)	One shared key, very fast — used for the audio stream
Asymmetric (RSA)	Public + private key pair, slow — used to deliver the AES key
Public key	Not secret; encrypts a message for its owner
Private key	Secret; the only key that can decrypt what the public key locked
Hybrid scheme	RSA delivers the AES key once; AES then encrypts all the audio
crypto_keys.h	C header with the RSA key numbers, generated on PC by keygen.py

Next step: run keygen.py on the PC to produce crypto_keys.h, then build the crypto.c / crypto.h module and self-test the RSA round-trip before wiring encryption into the live call.

End of Phase 7 — Encryption Explained.